

Import the libraries

```
In [72]: import pandas as pd  #loading and manipulating our dataset
import numpy as np  #numerical operations
import matplotlib.pyplot as plt  #Creating Charts
import seaborn as sns
```

Uploading the Dataset

```
In [73]: from google.colab import files
uploaded = files.upload()
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun cell to enable.

Saving Cars_Dataset.csv to Cars_Dataset (1).csv

Load the dataset

```
In [74]: df = pd.read_csv('/content/Cars_Dataset.csv')
df
```

Out[74]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel
0	1	3	alfa-romero giulia	gas	std	two	convertible	rwd
1	2	3	alfa-romero stelvio	gas	std	two	convertible	rwd
2	3	1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd
3	4	2	audi 1 0 0 ls	gas	std	four	sedan	fwd
4	5	2	audi 1 0 0 ls	gas	std	four	sedan	4 wd
...	...	...	...	...	...	...	...	...
2 0 3	2 0 4	- 1	volvo 2 4 6	diesel	turbo	four	sedan	rwd
2 0 4	2 0 5	- 1	volvo 2 6 4 gl	gas	turbo	four	sedan	rwd
2 0 5	2 1	0	chevrolet vega 2 3 0 0	gas	std	four	sedan	fwd
2 0 6	7 6	1	mercury cougar	gas	turbo	two	hatchback	rwd
2 0 7	1 5 1	1	toyota corona mark ii	gas	std	two	hatchback	fwd
2 0 8	rows ×	2 6	columns					

In [75]: `df.head()` *#this will show me the first 5 rows and all the N columns*

```
Out[75]:
```

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	engine
0	1	3	alfa-romero giulia	gas	std	two	convertible	rwd	
1	2	3	alfa-romero stelvio	gas	std	two	convertible	rwd	
2	3	1	alfa-romero Quadrifoglio	gas	std	two	hatchback	rwd	
3	4	2	audi 1 0 0 ls	gas	std	four	sedan	fwd	
4	5	2	audi 1 0 0 ls	gas	std	four	sedan	4 wd	

5 rows × 10 columns

Checking the Structure of the Dataset

```
In [76]: df.shape
```

```
Out[76]: (208, 26)
```

Checking all the Columns

```
In [77]: df.columns    #To get the list of all the columns
```

```
Out[77]: Index(['car_ID', 'symboling', 'CarName', 'fueltype', 'aspiration',
                'doornumber', 'carbody', 'drivewheel', 'engineLocation', 'wheelbase',
                'carlength', 'carwidth', 'carheight', 'curbweight', 'enginetype',
                'cylindernumber', 'enginesize', 'fuelsystem', 'boreRatio', 'stroke',
                'compressionratio', 'horsepower', 'peakrpm', 'citympg', 'highwaympg',
                'price'],
                dtype='object')
```

Complete Information regarding the Dataset

```
In [78]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 208 entries, 0 to 207
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   car_ID                 208 non-null    int64
1   symboling              208 non-null    int64
2   CarName                208 non-null    object
3   fueltype               207 non-null    object
4   aspiration              208 non-null    object
5   doornumber             208 non-null    object
6   carbody                208 non-null    object
7   drivewheel             208 non-null    object
8   enginelocation         208 non-null    object
9   wheelbase              208 non-null    float64
10  carlength              208 non-null    float64
11  carwidth               208 non-null    float64
12  carheight              208 non-null    float64
13  curbweight             208 non-null    int64
14  enginetype             208 non-null    object
15  cylindernumber         208 non-null    object
16  enginesize             207 non-null    float64
17  fuelsystem             208 non-null    object
18  boreratio              208 non-null    float64
19  stroke                 208 non-null    float64
20  compressionratio       208 non-null    float64
21  horsepower             207 non-null    float64
22  peakrpm                208 non-null    int64
23  citympg                208 non-null    int64
24  highwaympg             208 non-null    int64
25  price                  207 non-null    float64
dtypes: float64(10), int64(6), object(10)
memory usage: 42.4+ KB

```

Check the dataset statistically

```
In [79]: df.describe(include="object")    #for inspecting the categorical variable
```

```
Out[79]:
```

	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	enginelocation	enginetype
count	2 0 8	2 0 7	2 0 8	2 0 8	2 0 8	2 0 8	2 0 8	2 0 8
unique	1 4 7	2	2	2	5	3	2	7
top	peugeot 5 0 4	gas	std	four	sedan	fwd	front	ohc
freq	6	1 8 7	1 7 0	1 1 6	9 7	1 2 2	2 0 5	1 5 1

Data Quality Check

```
In [80]: df.isnull().sum()
```

Out[80]:

	0
car_ID	0
symboling	0
CarName	0
fueltype	1
aspiration	0
doornumber	0
carbody	0
drivewheel	0
enginelocation	0
wheelbase	0
carlength	0
carwidth	0
carheight	0
curbweight	0
enginetype	0
cylindernumber	0
enginesize	1
fuelsystem	0
boreratio	0
stroke	0
compressionratio	0
horsepower	1
peakrpm	0
citympg	0
highwaympg	0
price	1

dtype: int64

Checking for Duplicates

```
In [81]: df.duplicated().sum()
```

Out[81]: np.int64(3)

```
In [82]: df[df.isnull().any(axis=1)] #this shows which row is actually having the
```

Out[82]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	er
1 0	1 1	2	bmw 3 2 0 i	gas	std	two	sedan	rwd	
3 5	3 6	0	honda accord lx	gas	std	four	sedan	fwd	
8 0	8 1	3	mitsubishi mirage g 4	NaN	turbo	two	hatchback	fwd	
1 2 0	1 2 1	1	plymouth fury iii	gas	std	four	hatchback	fwd	

4 rows × 2 6 columns

```
In [83]: missing_values = df.isnull().sum() #We store it in a variable  
missing_values = missing_values[missing_values > 0]  
missing_values
```

Out[83]:

	0
fueltype	1
enginesize	1
horsepower	1
price	1

dtype: int 6 4

```
In [84]: missing_percentage = (df.isnull().sum() / len(df)) * 100 #This tells us  
missing_percentage = missing_percentage[missing_percentage > 0]  
missing_percentage
```

Out[84]:

	0
fueltype	0 . 4 8 0 7 6 9
enginesize	0 . 4 8 0 7 6 9
horsepower	0 . 4 8 0 7 6 9
price	0 . 4 8 0 7 6 9

dtype: float 6 4

```
In [85]: df[df.duplicated(keep=False)].sort_values(by="car_ID")
```

Out[85]:

	car_ID	symboling	CarName	fueltype	aspiration	doornumber	carbody	drivewheel
	2 0	2 1	0	chevrolet vega 2 3 0 0	gas	std	four	sedan fwd
	2 0 5	2 1	0	chevrolet vega 2 3 0 0	gas	std	four	sedan fwd
	7 5	7 6	1	mercury cougar	gas	turbo	two	hatchback rwd
	2 0 6	7 6	1	mercury cougar	gas	turbo	two	hatchback rwd
	1 5 0	1 5 1	1	toyota corona mark ii	gas	std	two	hatchback fwd
	2 0 7	1 5 1	1	toyota corona mark ii	gas	std	two	hatchback fwd

6 rows × 9 columns

Check categorical consistency

```
In [86]: categorical_columns = df.select_dtypes(include=object).columns  #for the categorical_columns
```

```
Out[86]: Index(['CarName', 'fueltype', 'aspiration', 'doornumber', 'carbody',  
               'drivewheel', 'enginelocation', 'enginetype', 'cylindernumber',  
               'fuelsystem'],  
              dtype='object')
```

```
In [87]: for column in categorical_columns:  
          print(f"\n--- {column} ---")  
          print(df[column].value_counts(dropna=False))
```

```
--- CarName ---
CarName
peugeot 504          6
toyota corolla       6
toyota corona        6
subaru dl            4
mitsubishi outlander 3
..
volkswagen super beetle 1
volkswagen rabbit custom 1
volvo 245             1
volvo diesel          1
volvo 246             1
Name: count, Length: 147, dtype: int64
```

```
--- fueltype ---
fueltype
gas      187
diesel   20
NaN       1
Name: count, dtype: int64
```

```
--- aspiration ---
aspiration
std      170
turbo    38
Name: count, dtype: int64
```

```
--- doornumber ---
doornumber
four     116
two       92
Name: count, dtype: int64
```

```
--- carbody ---
carbody
sedan      97
hatchback  72
wagon      25
hardtop     8
convertible 6
Name: count, dtype: int64
```

```
--- drivewheel ---
drivewheel
fwd      122
rwd       77
4wd        9
Name: count, dtype: int64
```

```
--- enginelocation ---
enginelocation
front     205
rear        3
Name: count, dtype: int64
```

```
--- enginetype ---
enginetype
ohc      151
ohcf      15
```

```

ohcv      13
dohc      12
l         12
rotor      4
dohcv      1
Name: count, dtype: int64

```

```

--- cylindernumber ---
cylindernumber
four      162
six       24
five      11
eight      5
two        4
twelve     1
three      1
Name: count, dtype: int64

```

```

--- fuelsystem ---
fuelsystem
mpfi      95
2bbl      68
idi       20
1bbl      11
spdi       9
4bbl       3
mfi        1
spfi       1
Name: count, dtype: int64

```

```

In [88]: before_cleaning = df.shape
print("Rows before cleaning:", before_cleaning[0])
print("Columns before cleaning:", before_cleaning[1])

```

```

Rows before cleaning: 208
Columns before cleaning: 26

```

Now Eliminating the Duplicate Rows

```

In [89]: print('Before removing the Duplicates',df.shape)
df = df.drop_duplicates()
print('After removing the Duplicates',df.shape) #We can Clearly see that

```

```

Before removing the Duplicates (208, 26)
After removing the Duplicates (205, 26)

```

Handle missing values

```

In [90]: df.isnull().sum() #Clearly we can see that ['fueltype','enginesize','hor

```

Out[90]:

	0
car_ID	0
symboling	0
CarName	0
fueltype	1
aspiration	0
doornumber	0
carbody	0
drivewheel	0
enginelocation	0
wheelbase	0
carlength	0
carwidth	0
carheight	0
curbweight	0
enginetype	0
cylindernumber	0
enginesize	1
fuelsystem	0
boreratio	0
stroke	0
compressionratio	0
horsepower	1
peakrpm	0
citympg	0
highwaympg	0
price	1

dtype: int64

```
In [91]: #Handle missing values in numerical columns
numerical_missing = ["horsepower", "price", "enginesize"]

for column in numerical_missing:
    df.loc[:, column] = df[column].fillna(df[column].median())
```

```
#Handle missing value in categorical column
df.loc[:, "fueltype"] = df["fueltype"].fillna(df["fueltype"].mode()[0])
```

```
In [92]: rows_after = df.shape[0]

print("\nDATA CLEANING - AFTER")
print("Rows:", rows_after)
print("Columns:", df.shape[1])
print("Missing cells:", df.isnull().sum().sum()) #By now we have removed
print("Duplicate rows:", df.duplicated().sum()) #By no we have removed
```

```
DATA CLEANING - AFTER
Rows: 205
Columns: 26
Missing cells: 0
Duplicate rows: 0
```

Displaying Numerical and Categorical Variables

```
In [93]: numerical_columns = df.select_dtypes(include=np.number).columns
categorical_columns = df.select_dtypes(include=object).columns
```

```
print("Numerical columns:")
print(numerical_columns)
print("\nCategorical columns:")
print(categorical_columns)
```

```
Numerical columns:
Index(['car_ID', 'symboling', 'wheelbase', 'carlength', 'carwidth',
      'carheight', 'curbweight', 'enginesize', 'bore ratio', 'stroke',
      'compressionratio', 'horsepower', 'peakrpm', 'citympg', 'highwaympg',
      'price'],
      dtype='object')

Categorical columns:
Index(['CarName', 'fueltype', 'aspiration', 'doornumber', 'carbody',
      'drivewheel', 'engine location', 'enginetype', 'cylindernumber',
      'fuelsystem'],
      dtype='object')
```

```
In [94]: df.dtypes
```

Out[94]: 0

car_ID	int 6 4
symboling	int 6 4
CarName	object
fueltype	object
aspiration	object
doornumber	object
carbody	object
drivewheel	object
enginelocation	object
wheelbase	float 6 4
carlength	float 6 4
carwidth	float 6 4
carheight	float 6 4
curbweight	int 6 4
enginetype	object
cylindernumber	object
enginesize	float 6 4
fuelsystem	object
boreratio	float 6 4
stroke	float 6 4
compressionratio	float 6 4
horsepower	float 6 4
peakrpm	int 6 4
citympg	int 6 4
highwaympg	int 6 4
price	float 6 4

dtype: object

Descriptive Statistics

In [95]:

```
display(df[numerical_columns].describe())
display(df[categorical_columns].describe())
```

	car_ID										symboling										wheelbase												
count	2	0	5	0	0	0	0	0	0	0	2	0	5	0	0	0	0	0	0	0	2	0	5	0	0	0	0	0	0	0	0		
mean	1	0	3	0	0	0	0	0	0	0	0.8	3	4	1	4	6					9	8	7	5	6	5	8	5		1	7	4	0
std			5	9	3	2	2	5	6	5	1.2	4	5	3	0	7					6	0	2	1	7	7	6		1	2	3		
min			1	0	0	0	0	0	0	0	-2	0	0	0	0	0	0				8	6	6	0	0	0	0	0	0	1	4	1	1
2 5 %			5	2	0	0	0	0	0	0	0	0	0	0	0	0	0				9	4	5	0	0	0	0	0	0	1	6	6	3
5 0 %	1	0	3	0	0	0	0	0	0	0	1	0	0	0	0	0	0				9	7	0	0	0	0	0	0	0	1	7	3	2
7 5 %	1	5	4	0	0	0	0	0	0	0	2	0	0	0	0	0	0				1	0	2	4	0	0	0	0	0	1	8	3	1
max	2	0	5	0	0	0	0	0	0	0	3	0	0	0	0	0	0				1	2	0	9	0	0	0	0	0	2	0	8	1

	CarName	fueltype	aspiration	doornumber	carbody	drivewheel	engine location	engine type	c
count	2 0 5	2 0 5	2 0 5	2 0 5	2 0 5	2 0 5	2 0 5	2 0 5	
unique	1 4 7	2	2	2	5	3	2	7	
top	peugeot 5 0 4	gas	std	four	sedan	fwd	front	ohc	
freq	6	1 8 5	1 6 8	1 1 5	9 6	1 2 0	2 0 2	1 4 8	

```
In [96]: print("Mean price:", df["price"].mean())    #Statistical data for better g
print("Median price:", df["price"].median())
print("Minimum price:", df["price"].min())
print("Maximum price:", df["price"].max())
print("Standard deviation:", df["price"].std())
```

Mean price: 13291.466668292682
Median price: 10320.0
Minimum price: 5118.0
Maximum price: 45400.0
Standard deviation: 7980.538829185835

Univariate Analysis

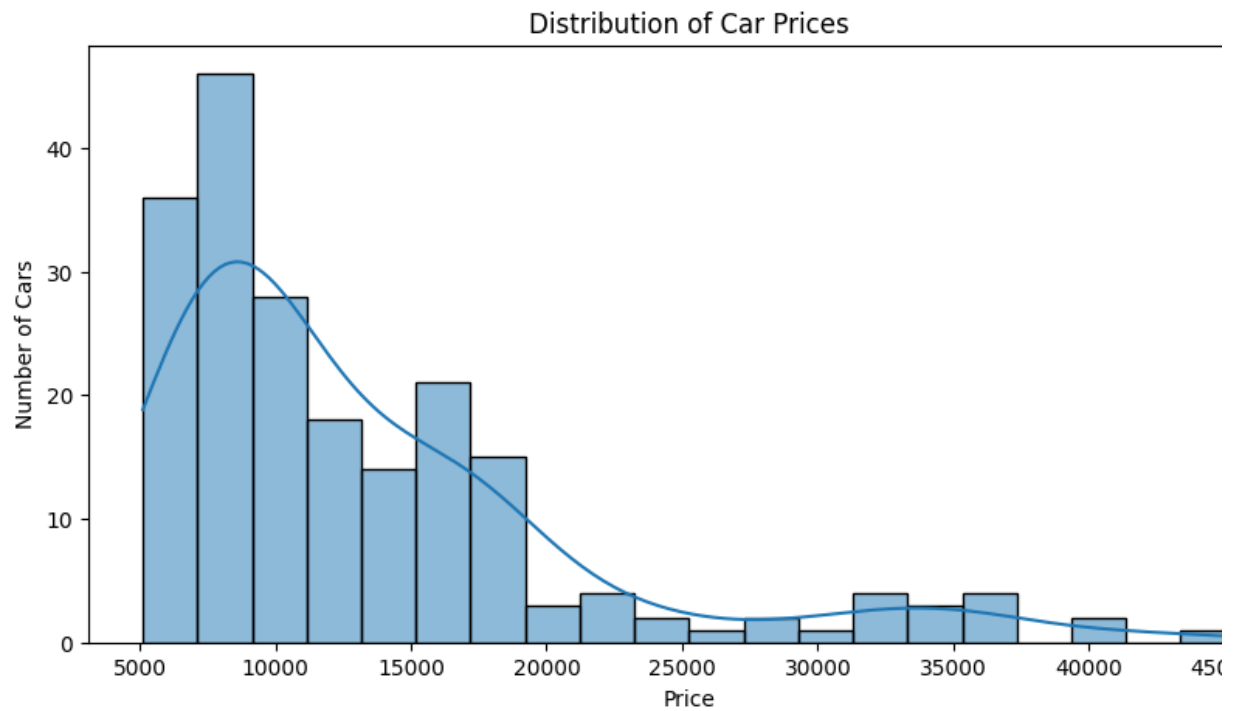
Histogram Plot

```
In [97]: plt.figure(figsize=(10, 5))

sns.histplot(data=df, x="price", bins=20, kde=True)    #to create a histogram

plt.title("Distribution of Car Prices")
plt.xlabel("Price")
plt.ylabel("Number of Cars")
```

```
plt.show()
```



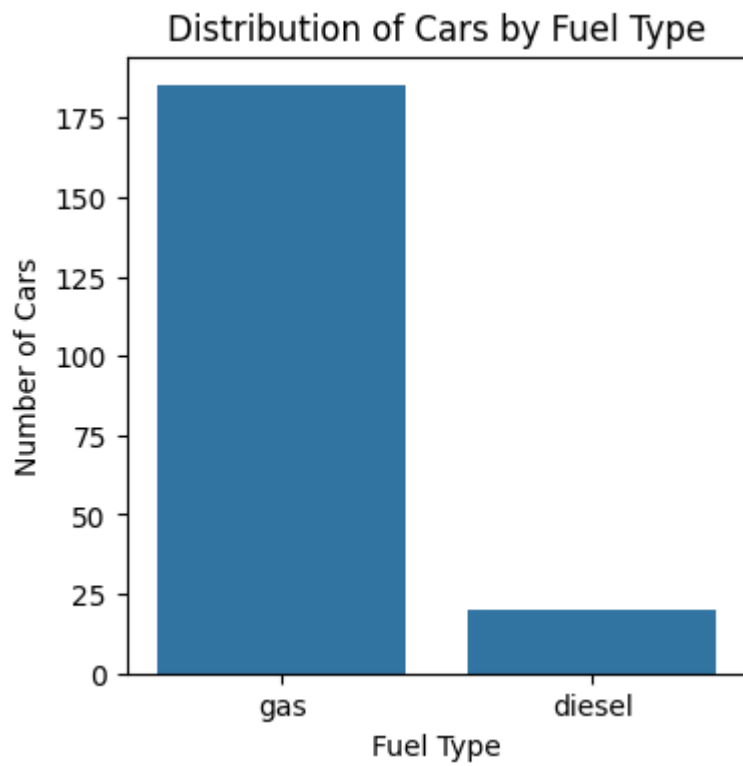
Count Plot

```
In [98]: plt.figure(figsize=(4, 4))

sns.countplot(data=df, x="fueltype")

plt.title("Distribution of Cars by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Number of Cars")

plt.show()
```



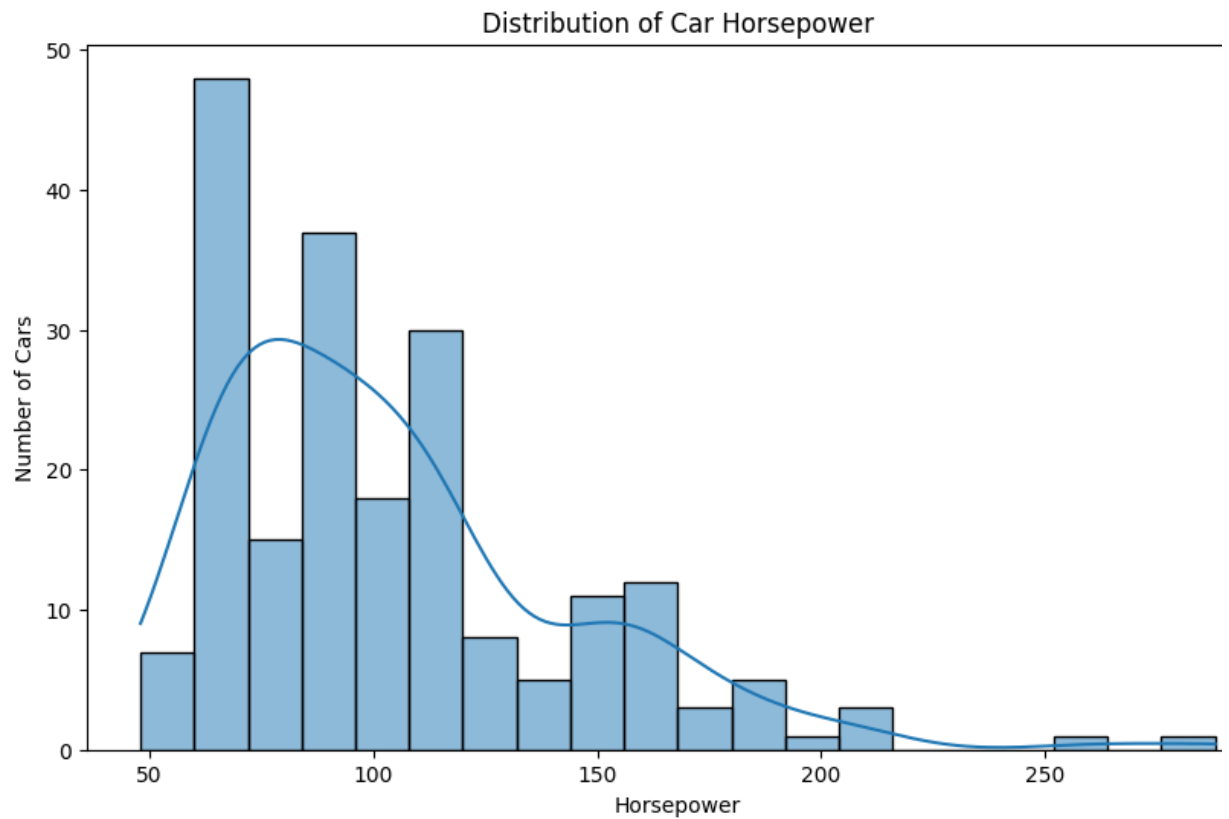
Histogram Plot

```
In [99]: plt.figure(figsize=(10, 6))

sns.histplot(data=df, x="horsepower", bins=20, kde=True)

plt.title("Distribution of Car Horsepower")
plt.xlabel("Horsepower")
plt.ylabel("Number of Cars")

plt.show()
```



Bivariate Analysis

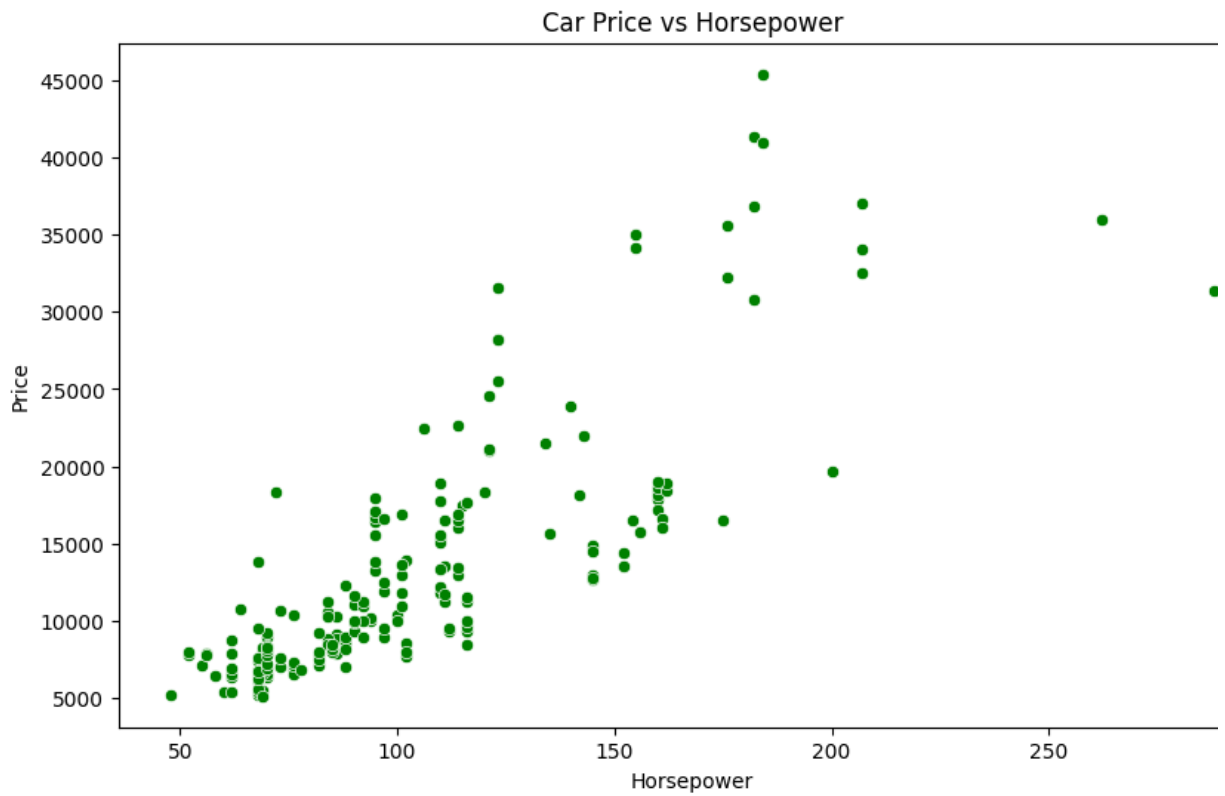
Scatter Plot

```
In [100... plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df, x="horsepower", y="price", color="green")

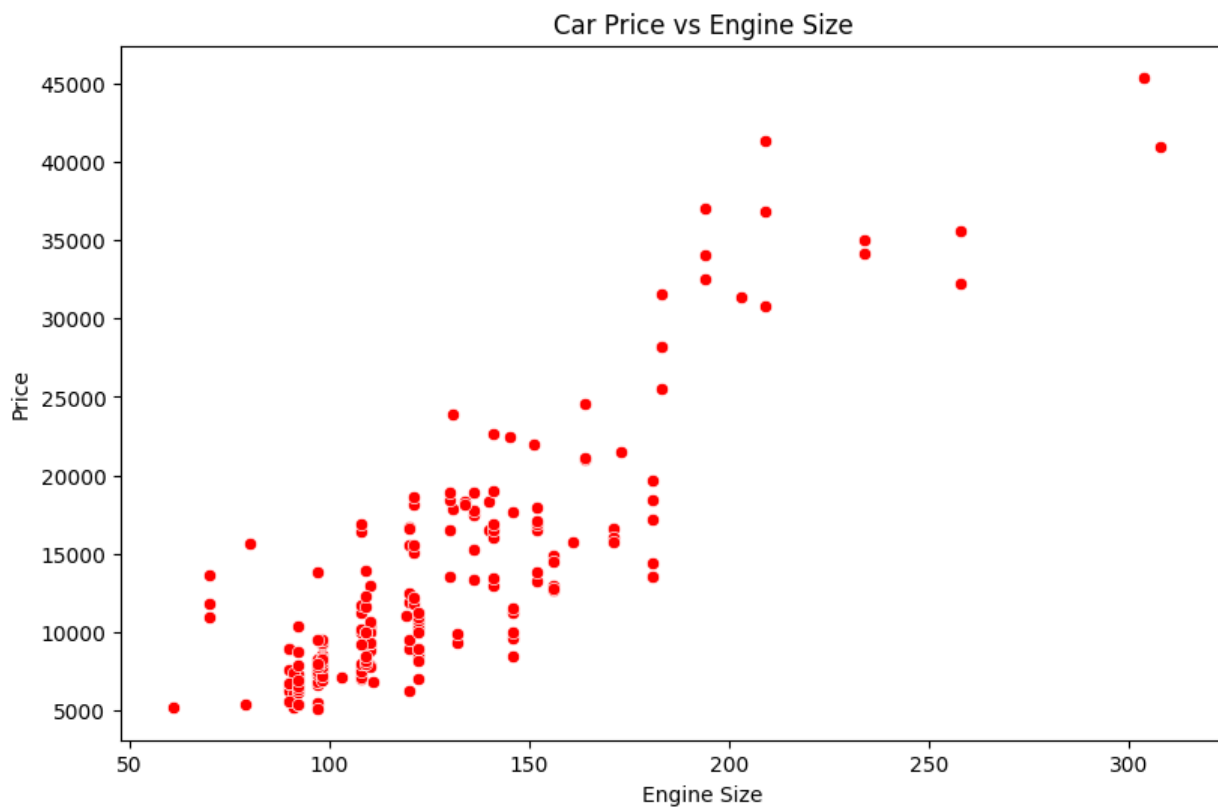
plt.title("Car Price vs Horsepower")
plt.xlabel("Horsepower")
plt.ylabel("Price")

plt.show()
```



Scatter Plot

```
In [101... plt.figure(figsize=(10,6))
sns.scatterplot(data=df,x="engine size",y="price",color="red")
plt.title("Car Price vs Engine Size")
plt.xlabel("Engine Size")
plt.ylabel("Price")
plt.show()
```



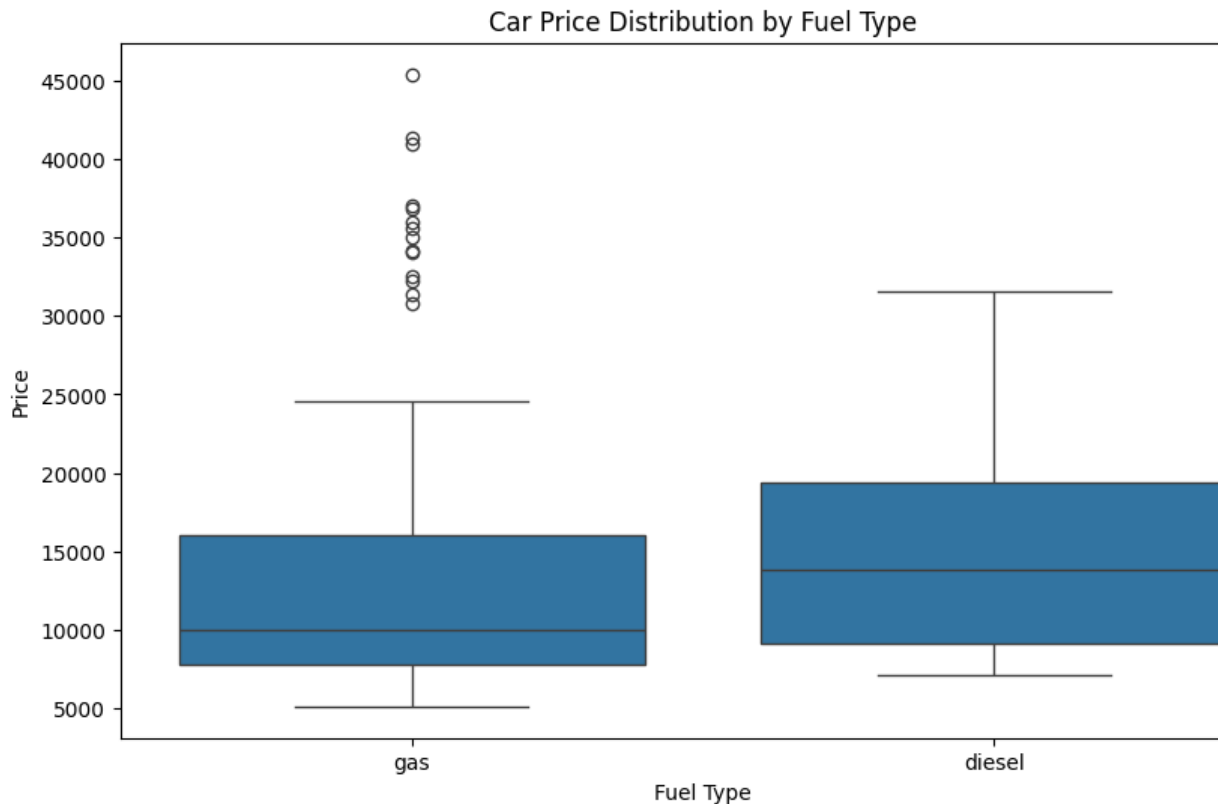
Box Plot

```
In [102... plt.figure(figsize=(10, 6))

sns.boxplot(data=df,x="fueltype",y="price")

plt.title("Car Price Distribution by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Price")

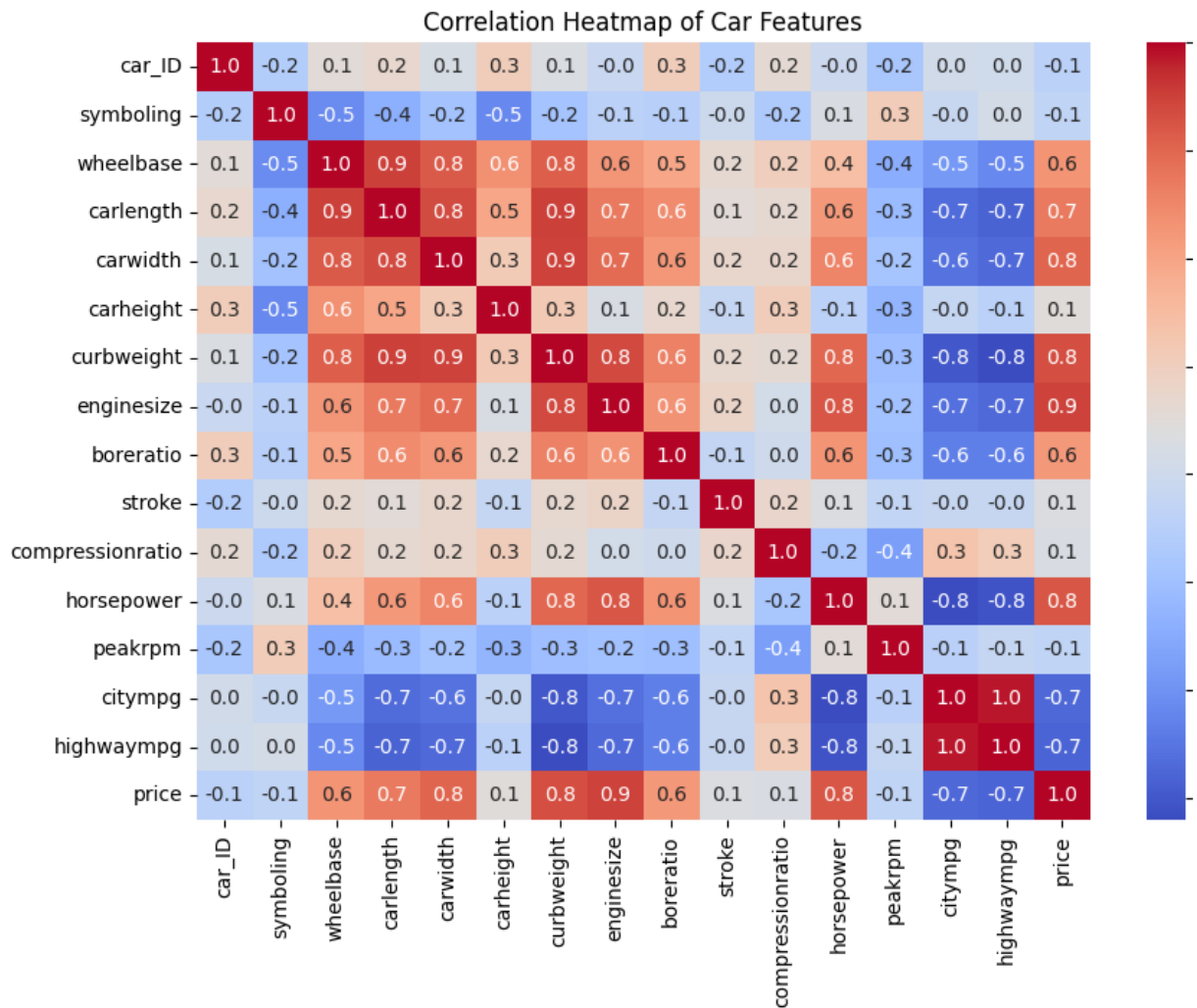
plt.show()
```



Multivariate Analysis

```
In [103... plt.figure(figsize=(10,7))
correlation = df[numerical_columns].corr()
sns.heatmap(
    correlation,  ##For Multiple variables we Use Corelation Heatmap
    annot=True,cmap="coolwarm",fmt=".1f")

plt.title("Correlation Heatmap of Car Features")
plt.show()
```



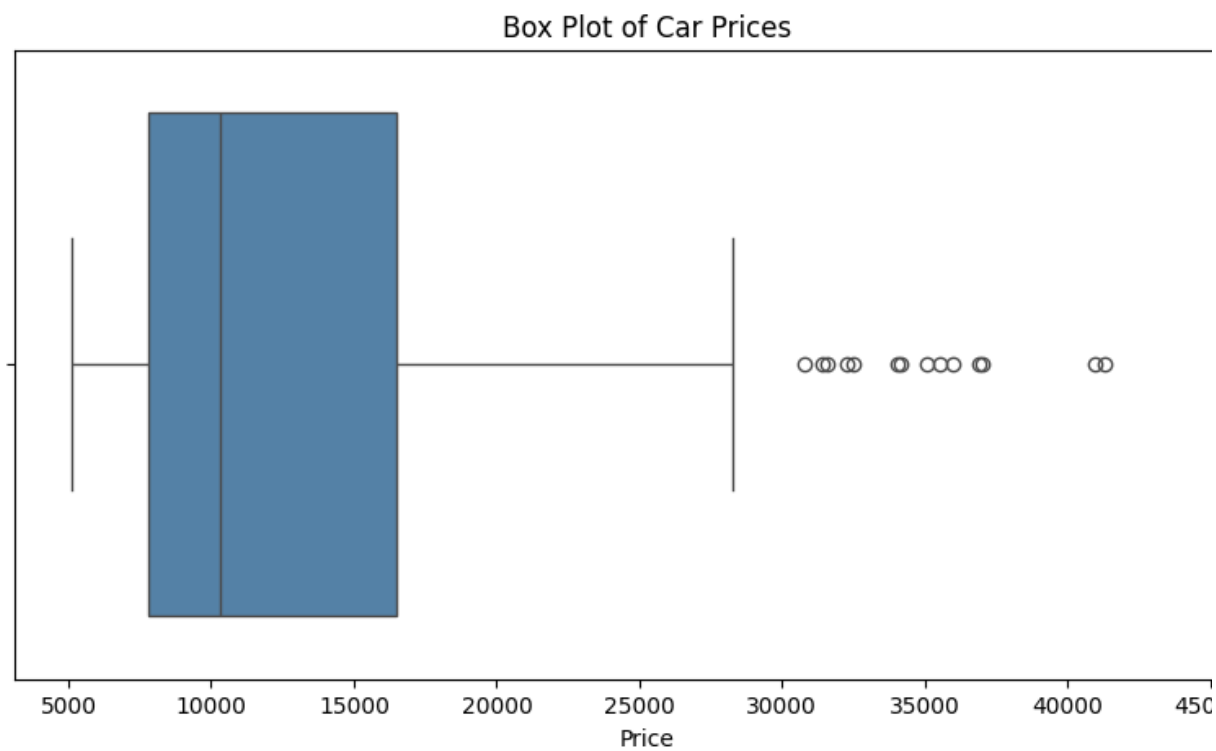
Outlier Analysis

```
In [104... plt.figure(figsize=(10, 5))

sns.boxplot(
    data=df,
    x="price",
    color="steelblue"
)

plt.title("Box Plot of Car Prices")
plt.xlabel("Price")

plt.show()
```



In [104...